



Software Requirements

Unit-II



- *Software Requirements*
- *Functional & Non-Functional Requirements.*
- *User Requirements.*
- *System Requirements.*
- *Interface specification.*
- *The Software requirement document.*



Software Requirements

- The requirement can be defined as a high-level abstract statement or a detailed mathematical functional specification of a system's services, functions, and constraints.
- They are depictions(Representation) of the characteristics and functionalities of the target system.
- Requirements denote the expectations of users from the software product.



- In the domain of **Software Engineering**, the concept of requirements plays a pivotal role.
- It guides the development processes and ensures that the end result aligns with **stakeholder's expectations** and **users' needs**.
- This makes understanding all different Requirements crucial in a software project.



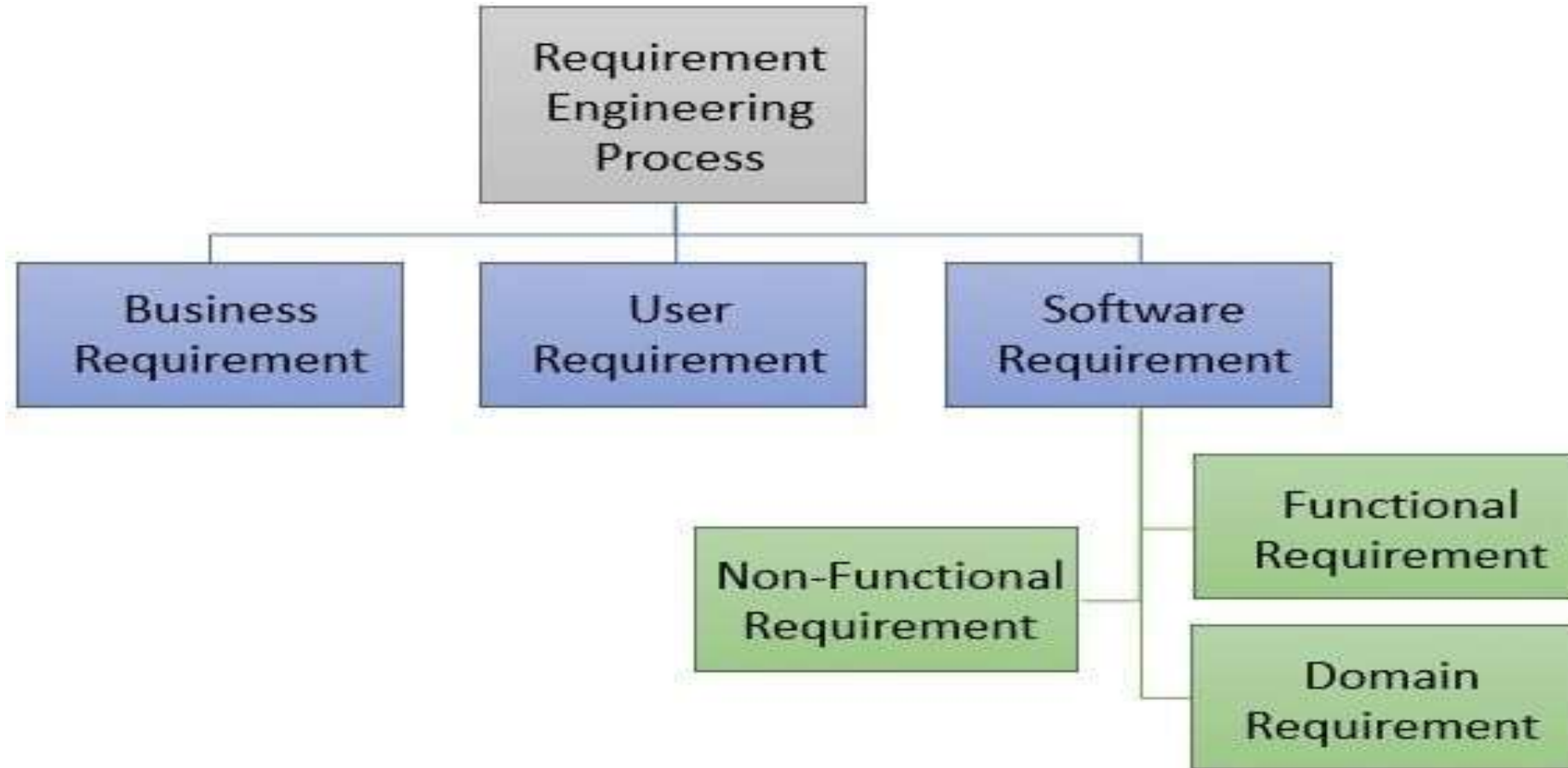
Bad Requirements



Good Requirements



Types of Requirements



User requirements:



- These requirements describe what the end-user wants from the software system.
- User requirements are usually expressed in **natural language** and are typically gathered through **interviews, surveys, or user feedback**.

Business requirements:

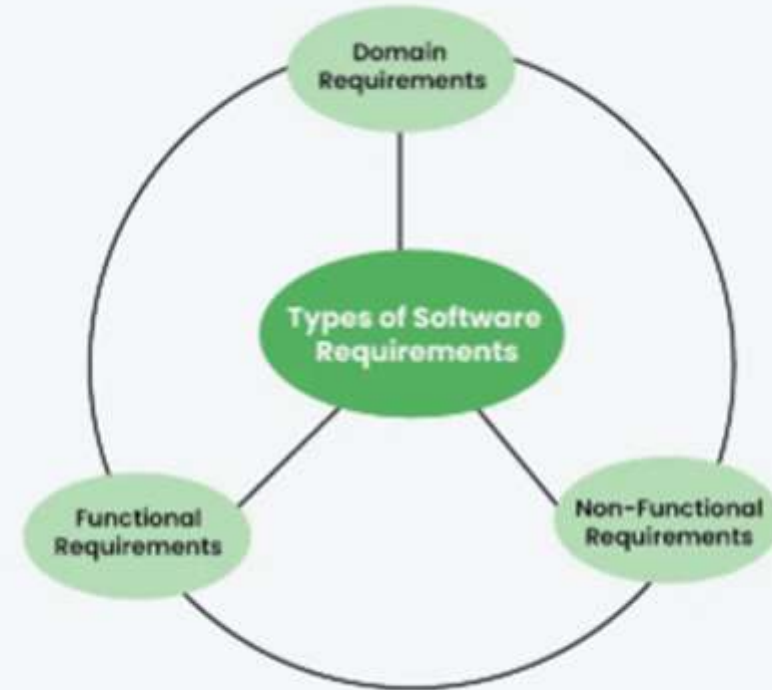


- These requirements describe the **business goals and objectives** that the software system is expected to achieve.
- Business requirements are usually expressed in terms of **revenue, market share, customer satisfaction, or other business metrics.**

There are three types of Software requirements as follows:



- *Functional requirements*
- *Non-Functional requirements*
- *Domain requirements*



Functional Requirements



- **Definition:**

- Functional requirements describe what the software should do and how it should interact with users or other systems.
- They define the **functions** or specific **features and operations** that the system must perform to meet business and user needs.
- Focus on system behavior and functionality.
- Represent the features that can be directly observed and tested in the final product.
- Functional requirements should be both complete and consistent
 - ✓ Completeness --All services required by the user should be defined
 - ✓ Consistent --Requirements should not have contradictory definition
- Difficult to achieve completeness and consistency for large system



- Examples:
- **User Authentication**: The system must allow users to log in using a username and password.
- **Search Functionality**: The software should enable users to search for products by name or category.
- **Report Generation**: The system should be able to generate sales reports for a specified date range.

Non-functional Requirements



- **Definition:**
- Non-functional requirements describe how the software performs a task rather than what it should do.
- Non-functional requirements (NFRs) define how a system should operate, focusing on performance, reliability, and user experience rather than specific features. They ensure the system is **efficient, secure, and maintainable** over time.
- Non-functional requirements are about the **system's behavior, quality, and constraints**.
- They ensure that the software meets certain standards of performance, usability, reliability, and security.



- Examples:
- 1) **Security**: Authorization, authentication and confidentiality.
- 2) **Performance**: speed and responsiveness
- 3) **Usability**: User interface design, accessibility, user experience.
- 4) **Reliability**: System stability, availability, error handling.
- 5) **Maintainability**: Ease of updates and fixes
- 6) **Portability**: Ability to run in different environments

Types of Non-Functional Requirements



1. Product Requirements

-Specify product behavior

-Include the following

- Usability
- Efficiency
- Reliability
- Portability



2. Organizational Requirements

--Derived from policies and procedures

--Include the following:

- Delivery
- Implementation
- Standard



3. External Requirements

--Derived from factors external to the system and its development process

--Includes the following

- Interoperability
- Ethical
- Legislative



Functional Requirements

- **Describes** what the system should do, i.e., specific functionality or tasks.
- **Focuses** on the behavior and features of the system.
- **Defines** the actions and operations of the system.
- **User authentication** data input/output, transaction processing

VS

Non Functional Requirements

- **Describes** how the system should perform, i.e., system attributes or quality.
- **Focuses** on the performance, usability, and other quality attributes.
- **Defines** constraints or conditions under which the system must operate
- **Scalability** security, response time, reliability, maintainability.

Example1: **Online Banking System**



- **Functional Requirements:**

- Users should be able to log in with their username and password.
- Users should be able to check their account balance.
- Users should receive notifications after making a transaction.

- **Non-functional Requirements:**

- The system should respond to user actions in less than 2 seconds.
- All transactions must be encrypted and comply with industry security standards.
- The system should be able to handle 100 million users with minimal downtime.



Example2: Food Delivery App

- **Functional Requirements:**

- Users can browse the menu and place an order.
- Users can make payments and track their orders in real time.

- **Non-functional Requirements:**

- The app should load the restaurant menu in under 1 second.
- The system should support up to 50,000 concurrent orders during peak hours.
- The app should be easy to use for first-time users, with an intuitive interface.

How to Gather Functional and Non-functional Requirements



- **Functional Requirements:**

- **Interviews:** Talk to stakeholders or users to understand their needs.
- **Surveys:** Distribute questionnaires to gather input from a larger audience.
- **Workshops:** Host sessions to brainstorm features and gather feedback.

- **Non-functional Requirements:**

- **Performance Benchmarks:** Consult with IT teams to set expectations for performance and load.
- **Security Standards:** Consult with security experts to define the best practices for data protection.
- **Usability Testing:** Test the system to find areas where users might struggle and refine the interface.

Domain Requirements



- Definition:
- Domain requirements are specific to the **domain or industry** in which the software operates.
- They include **terminology, rules, and standards** relevant to that particular domain.
- Domain requirements can be **functional or non-functional**.
- These are essential functions that a system of specific domains must necessarily exhibit.



- Examples:
- **Healthcare**: The software must comply with HIPAA (Health Insurance Portability and Accountability Act of 1996) regulations for handling patient data.
- **Finance**: The system should adhere to GAAP(Generally Accepted Accounting Principles) standards for financial reporting.
- **E-commerce**: The software should support various payment gateways like PayPal, Stripe, and credit cards.

User Requirements



- Should describe functional and non-functional requirements in such a way that they are understandable by system users who don't have detailed technical knowledge.
- User requirements are defined using natural language, tables and diagrams as these can be understood by all users.

Problems with Natural Language



- **Lack of clarity**
 - ✓ Precision is difficult without making the document difficult to read.
- **Requirements confusion**
 - ✓ Functional and non-functional requirements tend to be mixed-up.
- **Requirements amalgamation**
 - ✓ Several different requirements may be expressed together.

Requirement Problems



- Database requirements includes both conceptual and detailed information
 - ✓ Describes the concept of a financial accounting system that is to be included in LIBSYS
 - ✓ However, it also includes the detail that managers can configure this system - this is unnecessary at this level.
- Grid requirement mixes three different kinds of requirement
 - ✓ Conceptual functional requirement (the need for a grid);
 - ✓ Non-functional requirement (grid units);
 - ✓ Non-functional UI requirement (grid switching).
 - ✓ Structured presentation

Guidelines for writing requirements



- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use "shall" for mandatory requirements, "should" for desirable requirements.
- Use text highlighting to identify key parts of the requirement.

System Requirements



- System requirements are more detailed descriptions of the software system's functions, services, and operational constraints.
- The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented. It may be part of the contract between the system buyer and the software developers.
- System requirements are expanded versions of the user requirements that are used by software engineers as the starting point for the system design.
- They add detail and explain how the user requirements should be provided by the system.
- They may be used as part of the contract for the implementation of the system and should therefore be a complete and detailed specification of the whole system.



Software Requirements Document (SRD)/(SRS)

- *Introduction*
- *Need*
- *Characteristics*
- *Structure of SRS*



Introduction

- In software engineering, a **Software Requirements Document (SRD)**, also known as a **Software Requirements Specification (SRS)**, is a blueprint outlining the **functionalities, features, and constraints** of a software product, ensuring a clear understanding among stakeholders for successful development.





- A **Software Requirements Specification (SRS)** is a formal document that outlines the functional and non-functional requirements of a software system.
- It serves as a bridge between **stakeholders, developers, and testers** by providing a clear and structured description of what the software should do.





- The SRS document ensures that all parties have a common understanding of the system's **capabilities, constraints, and expected behavior**.
- It helps in reducing **misunderstandings**, improving **software quality**, and ensuring that the final product meets user expectations.

Need for Software Requirements Specification (SRS)



- 1. Clarity and Understanding** – Ensures that developers, clients, and stakeholders have a clear understanding of the software requirements.
- 2. Standardization** – Provides a standardized way of documenting software requirements, reducing ambiguities.
- 3. Efficient Development** – Serves as a reference document for developers, testers, and project managers, streamlining the software development process.
- 4. Improved Communication** – Acts as a contract between customers and developers, ensuring alignment in expectations.
- 5. Risk Reduction** – Identifies potential risks, constraints, and dependencies early in the development process.
- 6. Facilitates Testing** – Helps testers verify that the final software meets the specified requirements.
- 7. Better Maintenance & Scalability** – Serves as a reference for future modifications and expansions of the software.

Characteristics of a Good SRS



Software Requirement Specification





- 1. Correctness** – The SRS should accurately describe all the requirements of the system.
- 2. Completeness** – It should include all functional, non-functional, and external requirements.
- 3. Unambiguous** – Requirements should be clearly defined without confusion or multiple interpretations.
- 4. Consistency** – The requirements should not contradict each other.



5.**Verifiability** – Each requirement should be testable and measurable.

6.**Modifiability** – The document should be structured so that changes can be made easily.

7.**Traceability** – Each requirement should be traceable throughout the software development life cycle (SDLC).

8.**Feasibility** – The requirements should be realistic and achievable within the given constraints.



Structure of an SRS Document

1. Introduction

- **Purpose** – Explains the purpose and objectives of the software.
- **Scope** – Defines what the software will and will not do.
- **Definitions, Acronyms, and Abbreviations** – Provides explanations for technical terms.
- **References** – Lists related documents, research papers, or industry standards.
- **Overview** – Briefly describes the structure of the document.





2. Overall Description

- **Product Perspective** – Explains whether the software is standalone or part of a larger system.
- **Product Functions** – Lists key functionalities of the system.
- **User Characteristics** – Describes the target users and their level of technical expertise.
- **Constraints** – Includes any hardware, software, or regulatory constraints.
- **Assumptions and Dependencies** – Lists dependencies like third-party tools, APIs, or platforms



3. Specific Requirements

- **Functional Requirements** – Defines the system's expected behavior and functionalities.
- **Non-Functional Requirements** – Includes performance, security, reliability, usability, and scalability requirements.
- **External Interface Requirements** – Describes user interfaces, hardware, software, and communication interfaces.



4. Appendices

Includes additional information, diagrams, flowcharts, or use cases.

5. Index

- Defines technical terms used in the document.

Uses of SRS document



- Development team require it for developing product according to the need.
- Test plans are generated by testing group based on the describe external behavior.
- Maintenance and support staff need it to understand what the software product is supposed to do.
- Project manager base their plans and estimates of schedule, effort and resources on it.
- customer rely on it to know that product they can expect.
- As a contract between developer and customer in documentation purpose.



*Thank you
All the Best*